

Two-Factor Decomposition of Deterministic One-Counter Automata Is Undecidable

Alp Eren Bütün
Independent Researcher
Türkiye
github.com/alperenbutun

Abstract

A deterministic one-counter automaton (DOCA) presentation is *2-factor composite* if its language is the intersection of the languages of two DOCAs having strictly fewer control states. We prove that deciding this property is undecidable. The result reaches the smallest nontrivial intersection budget: the hardness does not rely on an unbounded family of factors, nor even on three factors.

The proof combines an elementary-abelian prime core with half-size quotient factors and an alternating-sentinel synchronization mechanism. A branch-labelled two-counter computation is encoded in two staggered phases. While one factor temporarily exposes its physical counter in order to perform an exact zero/decrement test, the other factor retains a positive sentinel and carries the missing lexical information; the roles then reverse. Consequently the two factors jointly enforce finite-control consistency and both source-counter trajectories, although neither factor stores both counters. If the source machine halts, a fixed suffix exposes a prime permutation-DFA section of full index. If it does not halt, two explicit quotient DOCAs, each with exactly half as many control states as the target, intersect to the target language exactly.

Undecidability already holds for complete real-time, ε -free DOCAs with one work symbol above a permanent bottom marker, all control states final, and counter updates in $\{-1, 0, +1\}$. The 2-factor-composite instances produced by the reduction form a non-recursively-enumerable set. Under the standard convention that factors need not be distinct, the same construction yields undecidability for every fixed factor budget $k \geq 2$.

Keywords. deterministic one-counter automata; automata decomposition; prime languages; state complexity; Minsky machines; permutation automata; undecidability.

1 Introduction

Intersection decomposition asks whether a language presentation can be replaced by several strictly smaller superlanguage presentations whose intersection recovers the original language. For deterministic finite automata (DFAs), Kupferman and Mosheiff introduced the corresponding notion of prime language and studied decidability, factor counts, structural witnesses, and permutation automata [2]. The bounded variant asks whether a presentation is the intersection of exactly k smaller factors. Jecker, Mazzocchi, and Wolf studied this parameter explicitly for DFAs; in particular, k -factor compositionality is NP-complete for commutative permutation DFAs, while the general DFA problem lies in PSPACE [1]. For unrestricted DFA primality, Spenner recently established NP-hardness [4].

Totzke asked for the analogous state-primality problem for deterministic one-counter automata [5]. Here “smaller” means fewer finite-control states than the presented DOCA, not a lower-dimensional counter resource. The one-counter setting is qualitatively different from the finite-state setting: exact verification of a guessed intersection decomposition cannot be reduced

to a finite product construction, and the problem statement already points to undecidability phenomena for intersections of DOCAs [5]. At the same time, equivalence of two given DOCAs over finite words is decidable [6], illustrating that decomposition introduces a different source of difficulty.

Our reduction begins from two independent ingredients that are developed completely below. First, an elementary-abelian translation core has a section rejecting only one point; that section is a prime permutation language of full state index. Second, a proposed two-counter computation is split into three predicates: one regular predicate for finite-control consistency and one deterministic one-counter predicate for each source counter. The problem is then to enforce all three predicates with only two smaller one-counter factors.

With three factors there is an immediate conceptual solution: use one regular-control factor and one factor for each source counter. The genuine boundary is therefore $k = 2$. Eliminating the third factor is not a matter of deleting a checker, because the regular-control checker also supplies lexical/syntax information when a counter checker temporarily reaches physical height zero. The alternating-sentinel construction below distributes that responsibility between the two counter factors themselves.

To the best of our knowledge, the state-smaller two-factor decomposition problem for DOCAs has not previously been settled. The contribution of this paper is a self-contained reduction in which two factors jointly replace the natural three-checker split. We encode each source transition in two staggered halves. During the first half, factor 1 exposes counter 1 while factor 2 remains positive and remembers the lexical transition label. During the second half, factor 2 exposes counter 2 while factor 1 remains positive and carries the lexical label. We call this *alternating-sentinel coupling*. It yields a first-deviation property: at every paired phase, once the next source transition is chosen, there is at most one next input label that both factors can process without poisoning. Hence the intersection itself enforces the syntax and the common witness.

Except for two standard external ingredients—undecidability of deterministic two-counter halting and the permutation-DFA primality theorem of Kupferman and Mosheiff—all constructions and reduction lemmas needed for the result are proved in this paper. In particular, no earlier Prime-DOCA construction is used as a black box.

Our main result is the following.

Theorem 1.1 (Main theorem). *The problem Factor-DOCA₂ is undecidable. Undecidability already holds when the input target and the two witnessing factors are complete real-time ε -free deterministic one-counter automata with one work symbol, a permanent bottom marker, all control states final, and counter updates in $\{-1, 0, +1\}$. In every composite instance produced by the reduction, each factor has exactly half as many control states as the target.*

The proof is a many-one reduction from halting of deterministic two-counter Minsky machines. In the halting branch the constructed target is actually prime against *arbitrary finite* intersections of smaller DOCAs, not merely against two factors. Thus the two sides of the reduction are sharply separated:

$$M \in \text{HALT}_2 \implies T_M \text{ is prime,} \quad M \notin \text{HALT}_2 \implies L(T_M) = L(F_1) \cap L(F_2).$$

The construction therefore shows that two factors already realize the full undecidability boundary.

2 Model and decomposition problem

A deterministic one-counter automaton has a finite control set Q , a permanent bottom marker, and one work symbol. We identify a configuration with a pair $(q, h) \in Q \times \mathbb{N}$, where h is the number of work symbols above the bottom marker. A transition reads either an input symbol or

ε , tests whether $h = 0$ or $h > 0$, changes the control state, and changes h by -1 , 0 , or $+1$, with decrements forbidden at height zero. We use the standard deterministic pushdown convention: from a fixed control/top situation there is at most one enabled ε -transition, and an enabled ε -transition excludes input-consuming transitions from the same situation.

Acceptance is by final control state and zero counter after the whole input is consumed. All automata constructed below are real-time and have every control state final, so for them acceptance is simply termination at height zero.

Definition 2.1 (*k-factor compositionality*). Let A be a DOCA with $n = |Q_A|$. For $k \geq 1$, call A *k-factor composite* if there are DOCAs $B_1, \dots, B_k$ such that

$$|Q_{B_i}| < n \quad (1 \leq i \leq k)$$

and

$$L(A) = \bigcap_{i=1}^k L(B_i).$$

The equality automatically implies $L(A) \subseteq L(B_i)$ for every factor. We write Factor-DOCA_k for the corresponding decision problem.

No distinctness condition is imposed on factors. This is the standard convention for k -factor intersection decomposition [1].

For comparison with Totzke's unrestricted problem, call a presented DOCA A *prime* if there is no finite family $B_1, \dots, B_t$ of DOCAs with $|Q_{B_j}| < |Q_A|$ and

$$L(A) = \bigcap_{j=1}^t L(B_j).$$

Thus primality quantifies over every finite factor budget, whereas Theorem 2.1 fixes the budget in advance.

3 Source machine, transcript, and finite prime core

3.1 A two-counter transcript

Let M be a deterministic two-counter Minsky machine with finite control Q_M , initial state p_0 , and two nonnegative counters c_1, c_2 ; halting from the zero configuration is undecidable [3]. We normalize M so that after its original halt instruction it deterministically drains c_1 to zero, then c_2 to zero, and enters a fresh terminal state p_f . Thus

$$M \text{ halts} \iff (p_f, 0, 0) \text{ is reached.} \quad (1)$$

Let Δ_M contain one symbol for every branch-labelled transition. For $\tau \in \Delta_M$, write $\text{src}(\tau)$ and $\text{tgt}(\tau)$ for its source and target control states, and

$$\text{op}_j(\tau) \in \{\text{inc}, \text{dec}, \text{zero}, \text{skip}\}$$

for the induced operation on counter j .

For $z = \tau_1 \cdots \tau_m \in \Delta_M^*$, define:

- $z \in R_M$ iff the labelled transitions form a control-consistent path from p_0 to p_f ;
- $z \in C_j$ iff starting from height 0, the operations $\text{op}_j(\tau_i)$ respect all zero/decrement guards and end again at height 0.

Lemma 3.1 (Transcript lemma). *For the normalized deterministic two-counter machine M ,*

$$\boxed{M \text{ halts} \iff R_M \cap C_1 \cap C_2 \neq \emptyset.}$$

Proof. A normalized halting run yields its branch-labelled transition word. Conversely, R_M fixes a control path. Inductively, C_1 and C_2 certify exactly the two guarded counter trajectories along that same path: a zero-labelled branch is admitted exactly at counter value zero, and a decrement-labelled branch exactly at positive value. Hence every labelled transition is genuinely enabled and the path reaches $(p_f, 0, 0)$, which is equivalent to halting by (1). $\square$

3.2 The elementary-abelian prime core

Choose a power of two $N = 2^d$ and let

$$V = (\mathbb{Z}_2)^d, \quad |V| = N.$$

Let $E = \{e_1, \dots, e_d\}$ be an alphabet whose letters act on V by translations $v \mapsto v + e_i$. Starting at 0, define

$$P_V = \{x \in E^* : \delta(0, x) \neq 0\}.$$

Lemma 3.2 (Prime core). *The translation DFA for P_V is minimal of index N , is a permutation DFA, has exactly $N-1$ accepting states, and P_V is prime with respect to intersection decomposition into smaller DFAs.*

Proof. Every vector is reachable. If $u \neq v$, choose a core word whose total translation is u . Appending it sends u to 0 and v to $v + u \neq 0$, so the states are distinguishable and the index is N . Each core letter is a permutation and only 0 rejects. The primality conclusion is the permutation-DFA theorem of Kupferman and Mosheiff: an index- N permutation DFA with $N-1$ accepting states is prime [2]. $\square$

3.3 Zero-cut extraction

The prime direction must exclude smaller factors even if they use ε -moves. The following section lemma supplies the required finite-state cut and is proved here for completeness.

Lemma 3.3 (Zero-cut section lemma). *Let B be a deterministic one-counter automaton, possibly with ε -moves, such that $E^* \subseteq L(B)$. For every fixed continuation u , the section*

$$S_u(B) = \{x \in E^* : xu \in L(B)\}$$

is recognized by a DFA with at most $|Q_B|$ states.

Proof. For every $x \in E^*$, run B on x and then follow its unique post-input ε -trajectory until the first accepting zero configuration is reached. Such a configuration exists because $x \in L(B)$. Determinism and ε /input exclusivity make this physical configuration a forced cut before every longer continuation: once the run has reached $(q, 0)$ after x , future behavior depends only on q and the unread input.

Use the set of control states occurring at these cuts as DFA states. A core letter induces a well-defined transition between cut states, and a cut state is accepting exactly when B , started from the corresponding physical zero configuration, accepts the fixed continuation u . Thus the induced DFA recognizes $S_u(B)$ and has at most $|Q_B|$ states. $\square$

4 Phase coding and alternating sentinels

The three-factor construction assigns R_M, C_1, C_2 to three separate factors. To use only two factors we must let the factors share the syntax/control burden without ever requiring one DOCA to store both unbounded counters.

For every $\tau \in \Delta_M$, introduce six phase-labelled symbols

$$O_\tau^1, A_\tau^1, C_\tau^1, O_\tau^2, A_\tau^2, C_\tau^2.$$

For $z = \tau_1 \cdots \tau_m$, let

$$\hat{z} = \prod_{\ell=1}^m O_{\tau_\ell}^1 A_{\tau_\ell}^1 C_{\tau_\ell}^1 O_{\tau_\ell}^2 A_{\tau_\ell}^2 C_{\tau_\ell}^2. \quad (2)$$

Add a query-start marker @ and terminal symbols $T_1, Z_1, T_2, Z_2, \$$. The canonical complete query is

$$u(z) = @ \hat{z} T_1 Z_1 T_2 Z_2 $. \quad (3)$$

Both checkers represent a source counter value c_j at stable points by physical height

$$h_j = c_j + 1.$$

The extra unit is a sentinel. To test a source zero/decrement guard, checker j temporarily removes its sentinel, making the physical height exactly c_j . The construction staggers these exposures: checker 2 remains positive while checker 1 is exposed, and checker 1 remains positive while checker 2 is exposed.

4.1 Checker 1

Checker 1 carries the source-control state and verifies counter 1. Its logical roles are

$$P_p \ (p \in Q_M), \quad K_\tau^A, K_\tau^C \ (\tau \in \Delta_M), \quad U_1, U_2, U_3, \quad \perp_1,$$

together with one raw/special role S_1 .

At $(S_1, 0)$, @ starts the checker in $(P_{p_0}, 1)$. From (P_p, h) with $h > 0$, the symbol O_τ^1 is legal only when $\text{src}(\tau) = p$; it removes the sentinel and moves to raw role S_1 . At raw S_1 , A_σ^1 performs $\text{op}_1(\sigma)$ on the exposed height, with illegal zero/decrement branches entering permanent-positive poison. A symbol C_ρ^1 restores the sentinel and moves to $P_{\text{tgt}(\rho)}$.

During the second half, checker 1 becomes the lexical witness: O_τ^2 moves from P_p to K_τ^A , only A_τ^2 moves to K_τ^C , and only C_τ^2 returns to $P_{\text{tgt}(\tau)}$. The height stays positive throughout this half.

4.2 Checker 2

Checker 2 verifies counter 2 and carries the complementary lexical memory. Its logical roles are

$$B, \quad A_\tau, C_\tau \ (\tau \in \Delta_M), \quad W_1, V_2, \quad \perp_2,$$

together with raw/special role S_2 .

At $(S_2, 0)$, @ starts $(B, 1)$. From B , O_τ^1 stores τ in A_τ ; only A_τ^1 reaches C_τ . The symbol C_τ^1 preserves that lexical role, and only O_τ^2 then removes checker 2's sentinel and enters raw S_2 . At raw S_2 , A_σ^2 performs $\text{op}_2(\sigma)$, and C_ρ^2 restores the sentinel and returns to B . Checker 1 forces equality of the second-half labels.

The apparent reuse of C_τ before and after C_τ^1 causes no ambiguity: if C_τ^1 is omitted, checker 1 is still raw and rejects a premature O_τ^2 .

4.3 Terminal block

From P_p , checker 1 permits T_1 only for $p = p_f$, removes its sentinel, and enters raw S_1 . The next Z_1 is legal only at height zero; it restores the sentinel and enters U_1 . Checker 1 then reads T_2, Z_2 through U_1, U_2, U_3 and finally removes the restored sentinel on $$$.

Checker 2 reads T_1, Z_1 through B, W_1, B , then T_2 removes its sentinel and enters raw S_2 . The next Z_2 is legal only at height zero; it restores the sentinel in V_2 , and $$$ removes it. Hence simultaneous terminal success is equivalent to source control p_f and both source counters zero.

5 Synchronization lemmas

Define a synchronized stable pair

$$\mathcal{G}(p, a, b) = ((P_p, a + 1), (B, b + 1)).$$

Lemma 5.1 (Block synchronization). *Suppose the two checkers start in $\mathcal{G}(p, a, b)$, consume a non-poisoning source-transition macro, and next return to a synchronized stable pair. Then the consumed macro is uniquely*

$$O_\tau^1 A_\tau^1 C_\tau^1 O_\tau^2 A_\tau^2 C_\tau^2$$

for a single $\tau \in \Delta_M$ with $\text{src}(\tau) = p$. Both source-counter guards are legal at (a, b) , and the new pair is

$$\mathcal{G}(\text{tgt}(\tau), a', b'),$$

where (a', b') are exactly the source-counter values after τ .

Proof. Checker 1 source-checks the opening O_τ^1 . Checker 2 stores that same label and forces A_τ^1, C_τ^1 , so checker 1 performs exactly the counter-1 operation of τ . Checker 1 rejects O^2 while still raw, hence the closing C_τ^1 cannot be omitted. Checker 2 then permits only O_τ^2 and exposes counter 2. Checker 1 stores the same label and forces A_τ^2, C_τ^2 , so checker 2 performs exactly the counter-2 operation. Both sentinels are restored and checker 1's control role is $P_{\text{tgt}(\tau)}$. $\square$

Lemma 5.2 (Common-legal-symbol property). *Fix a non-poison paired phase after the opening @ and before the first simultaneous zero return. Apart from the choice of the next source transition τ at a stable source boundary, there is at most one next phase label that both checkers can process without poison. For a chosen τ , the common sequence is forced to be*

$$O_\tau^1 \rightarrow A_\tau^1 \rightarrow C_\tau^1 \rightarrow O_\tau^2 \rightarrow A_\tau^2 \rightarrow C_\tau^2.$$

At a terminal boundary it is forced to be

$$T_1 \rightarrow Z_1 \rightarrow T_2 \rightarrow Z_2 \rightarrow \$.$$

Proof. This follows by inspecting the paired roles. After O_τ^1 , checker 2's state A_τ forces A_τ^1 . Next C_τ permits C_τ^1 , while checker 1 rejects a premature O^2 . Once C_τ^1 restores checker 1's sentinel, checker 2 still remembers τ and forces O_τ^2 . Checker 1 then forces A_τ^2 and C_τ^2 . The terminal roles force the terminal sequence analogously. A core letter during an exposed raw-zero phase may be non-poisoning for the exposed checker, but the complementary checker is positive and therefore poisons on that same core letter; hence it is never a common legal continuation. $\square$

Lemma 5.3 (Alternating sentinel). *After the opening @ and before the final common reset \$, every proper non-poison prefix leaves at least one of the two checker counters strictly positive.*

Proof. Checker 1 removes its sentinel only during the $O^1 A^1$ interval, while checker 2 keeps its sentinel. The C^1 symbol restores checker 1's sentinel before checker 2 removes its sentinel on O^2 . During $O^2 A^2$, checker 1 therefore remains positive, and C^2 restores checker 2's sentinel. The terminal block is staggered in the same order: $T_1 Z_1$ opens and recloses checker 1 before $T_2 Z_2$ opens checker 2. Only the final \$ removes both restored sentinels on the same input symbol. $\square$

Lemma 5.4 (First simultaneous zero return). *Start both checkers in their special zero configurations and consume an opening @. A later prefix is the first prefix after that opening at which both checkers are again at height zero without poison if and only if that prefix is*

$$u(z) = @ \hat{z} T_1 Z_1 T_2 Z_2 \$$$

for some $z \in R_M \cap C_1 \cap C_2$.

Proof. By Theorem 5.2, every common non-poison continuation is forced block by block. By Theorem 5.1, each complete block realizes one genuinely enabled source transition and updates the two represented counters correctly. By Theorem 5.3, no proper prefix before the final common reset can have both counters zero. The terminal block succeeds for both checkers exactly at source state p_f with both source counters zero. Conversely, any normalized halting transcript follows all these transitions. $\square$

Combining Theorems 3.1 and 5.4 gives

$$\boxed{M \text{ halts} \iff \text{a first simultaneous zero-return query exists.}} \quad (4)$$

6 Target and two half-size factors

Let

$$q = |Q_M|, \quad m = |\Delta_M|.$$

Choose a power of two $N = 2^d$ satisfying

$$N/2 \geq q + 2m + 5. \quad (5)$$

Choose distinct nonzero vectors $r_1, r_2 \in V$ and put

$$H_i = \{0, r_i\}, \quad Q_i = V/H_i, \quad S_i = H_i.$$

Thus $|Q_i| = N/2$.

Checker 1 needs at most $q + 2m + 5$ control names: q source-control roles, $2m$ lexical roles, three terminal roles, one poison role, and one raw/special role. Checker 2 needs at most $2m + 5$ names. Hence (5) packs both checkers into their quotient state sets. The target can use disjoint positive-role families of total size at most $q + 4m + 10 \leq N$. The same underlying state names may carry unrelated zero-height core meanings and positive-height checker meanings; physical height disambiguates them, so no extra control states are needed.

6.1 The target T_M

The target state set is exactly V , every state is final, and the input alphabet is the disjoint union of the core alphabet E and the phase/query alphabet.

At height zero, every core letter acts by translation. At exact anchor 0, every non-core symbol enters permanent-positive poison. At r_i , the special zero table of checker i applies. Every other nonzero zero-height state is generic: all non-reset query symbols leave the target at the same zero anchor, while the final reset $\$$ sends it to 0. At positive height, the packed logical role determines the appropriate checker transition; every unspecified positive transition enters permanent-positive poison.

A core symbol read at height zero always performs core translation, even if the target has reached r_i as a temporary raw-zero checker configuration. A core symbol read at positive height poisons. Thus no hidden query-mode bit is assumed at height zero.

6.2 The factors F_1, F_2

Factor F_i has control set $Q_i = V/H_i$ and all states final. At height zero, core letters act by quotient translation:

$$(v + H_i, 0) \xrightarrow{e} (v + e + H_i, 0).$$

At the special coset S_i , checker i 's special/raw transition table applies. Every non-special zero coset deliberately overaccepts query symbols: non-reset query symbols preserve that zero coset, while $\$$ resets to S_i . Positive logical roles and a permanent-positive poison role are packed into existing quotient state names. No additional state is introduced.

Lemma 6.1 (Size and syntactic restrictions). *T_M is a complete real-time ε -free DOCA with exactly N control states. Each F_i is a complete real-time ε -free DOCA with exactly $N/2$ control states. All three automata use one work symbol, have all states final, and use only updates in $\{-1, 0, +1\}$.*

Proof. The state counts follow from the construction and (5). Each transition case is determined by the current control role, zero/positive status, and input class. Undefined cases are totalized into an existing permanent-positive poison role, using $+1$ at zero and 0 at positive height. Hence no sink state or ε -transition is needed. $\square$

7 The quotient invariant and forward inclusion

Lemma 7.1 (Quotient invariant). *Fix $i \in \{1, 2\}$. Whenever, after a common consumed prefix, the target is at zero height in exact core state v , factor F_i is at*

$$(v + H_i, 0),$$

unless an earlier target rejection has already made the implication irrelevant.

Proof. Core letters preserve the relation by translation. At a generic nonzero anchor, the target and every non-special factor remain at their zero states throughout non-reset query symbols; a successful reset sends the target to 0 and the factor to H_i .

At owner anchor r_i , factor F_i is special and executes exactly the same checker transitions as the target. If a raw counter action returns to height zero before the query is complete, both return to the same owner configuration r_i/H_i . At the other active anchor r_j , factor F_i is non-special and remains at zero in coset $r_j + H_i$ while the target runs checker j ; whenever checker j temporarily returns to raw zero, its exact state is again r_j , so the relation still holds.

If a core letter interrupts an exposed raw-zero phase, the target translates from the exact owner anchor and every zero-height factor performs the corresponding quotient translation. Finally, a non-core symbol from exact anchor 0 makes the target permanently positive and creates no later zero-height obligation. These cases exhaust all ways to pass between zero-height target configurations. $\square$

Lemma 7.2 (Forward inclusion). *For $i = 1, 2$,*

$$L(T_M) \subseteq L(F_i).$$

Proof. An accepted target word ends at zero height in some exact state v . By Theorem 7.1, factor F_i ends at zero height in quotient state $v + H_i$. Since every factor state is final, F_i accepts. $\square$

8 Halting exposes a prime section

Assume M halts and choose $z \in R_M \cap C_1 \cap C_2$. Fix the suffix $u = u(z)$ from (3). For a core word $x \in E^*$ let $v = \delta(0, x)$.

If $v = 0$, the first $@$ enters permanent-positive poison. If $v = r_1$, checker 1 successfully processes u ; if $v = r_2$, checker 2 does so. At every other nonzero anchor the query is deliberately overaccepted and the final $\$$ resets the target to zero. Hence

$$\boxed{xu \in L(T_M) \iff \delta(0, x) \neq 0.} \tag{6}$$

The fixed-suffix section is exactly P_V .

Lemma 8.1 (Halting implies primality). *If M halts, then T_M is prime: there is no finite family of DOCAs $B_1, \dots, B_t$ with $|Q_{B_j}| < N$ and*

$$L(T_M) = \bigcap_{j=1}^t L(B_j).$$

Proof. Suppose such a family exists. Every core word belongs to $L(T_M)$, hence to every $L(B_j)$. Apply Theorem 3.3 to each B_j with the fixed suffix u . This yields DFAs D_j with

$$|Q_{D_j}| \leq |Q_{B_j}| < N$$

and, by (6),

$$P_V = \bigcap_{j=1}^t L(D_j).$$

This contradicts the DFA-primality of P_V from Theorem 3.2. $\square$

9 Nonhalting gives an exact two-factor decomposition

Assume from now on that M does not halt. By (4), no query launched with both factors special can have a first simultaneous zero return.

Lemma 9.1 (Reverse inclusion). *If M does not halt, then*

$$L(F_1) \cap L(F_2) \subseteq L(T_M).$$

Proof. Take $w \notin L(T_M)$. Since the target is complete, real-time, and all states are final, its run on w ends at positive height. Let a be the last zero-height exact target state before the final positive episode.

If $a = r_i$, then by Theorem 7.1 factor F_i is special at that point and has the same physical transition table as the target checker. The target never returns to zero afterward, so the owner factor also remains positive and rejects. This includes the case where $a = r_i$ is a temporary raw-zero configuration reached inside a malformed or unfinished checker episode.

A generic nonzero a is impossible: generic non-core behavior never creates positive height.

It remains to consider $a = 0$. Both factors are then simultaneously special. A non-core symbol different from @ sends at least one special factor to permanent-positive poison; in particular, \$ is not an immediate reset from a special zero state. Suppose the final positive episode begins with @. By Theorem 5.2, a wrong phase, label mismatch, repeated marker, or premature terminal symbol poisons at least one factor. An illegal source zero/decrement branch poisons the exposed checker. A core symbol during an exposed raw-zero interval translates the exposed checker at height zero, but the complementary checker is still positive and therefore enters permanent-positive poison. If the input ends before a common reset, Theorem 5.3 leaves at least one factor positive. Finally, if both factors were to return to zero, their first simultaneous zero return would contradict nonhalting by Theorem 5.4. Thus at least one factor rejects w . $\square$

Combining Theorems 7.2 and 9.1 gives the exact decomposition.

Lemma 9.2 (Two factors suffice in the nonhalting case). *If M does not halt, then*

$$\boxed{L(T_M) = L(F_1) \cap L(F_2)}, \quad |Q_{F_1}| = |Q_{F_2}| = N/2 < N.$$

10 Undecidability and consequences

Theorem 10.1 (Two-factor undecidability). *The mapping $M \mapsto T_M$ is computable and satisfies*

$$M \text{ halts} \implies T_M \text{ is prime,}$$

while

$$M \text{ does not halt} \implies L(T_M) = L(F_1) \cap L(F_2)$$

for two explicit factors with $N/2$ states each. Consequently Factor-DOCA₂ is undecidable, even under the restrictions of Theorem 6.1.

Proof. The halting implication is Theorem 8.1; the nonhalting implication is Theorem 9.2. A decider for Factor-DOCA₂ would therefore decide halting of deterministic two-counter machines. $\square$

Corollary 10.2 (Unrestricted Prime-DOCA). *It is undecidable whether a presented DOCA is prime in the unrestricted finite-intersection sense. The same restricted target class suffices.*

Proof. The reduction of Theorem 10.1 maps halting machines to prime targets and nonhalting machines to targets that already decompose into two smaller factors. Hence a decider for unrestricted primality would decide deterministic two-counter halting. $\square$

Corollary 10.3 (Every fixed budget $k \geq 2$). *For every fixed integer $k \geq 2$, Factor-DOCA_k is undecidable under the same restrictions.*

Proof. The case $k = 2$ is Theorem 10.1. For $k > 2$, repeat either of the two factors. Repetition preserves the intersection language and the standard k -factor definition does not require distinct factors. $\square$

Corollary 10.4 (Bounded-factor input). *The problem that receives a DOCA A together with an integer k and asks whether A is k -factor composite is undecidable, even when every reduction instance has $k = 2$.*

Corollary 10.5 (Non-semidecidability). *The set of 2-factor-composite DOCA presentations is not recursively enumerable.*

Proof. The construction gives

$$M \notin \text{HALT}_2 \iff T_M \in \text{Factor-DOCA}_2.$$

Deterministic two-counter nonhalting is not recursively enumerable, so recursive enumerability of Factor-DOCA₂ would give a recursive enumeration of nonhalting instances by computable preimage. $\square$

11 Interpretation

The result identifies the exact nontrivial factor-budget boundary for this decomposition problem. Unrestricted primality permits an arbitrary finite number of smaller factors, but the present theorem shows that unbounded factor supply is not the source of the hardness. Two factors, the smallest budget capable of expressing a genuine intersection decomposition, already support undecidability.

The finite prime obstruction and the infinite-state synchronization play different roles. The elementary-abelian core handles the halting branch by exposing a fixed regular section that no intersection of smaller finite-state sections can realize. The alternating-sentinel code handles the nonhalting branch by making two one-counter factors jointly certify a two-counter transcript.

Neither factor stores two unbounded counters. Instead, each factor stores lexical information exactly while the other temporarily gives up its sentinel to expose an exact zero test. In this sense, the natural third regular checker has not been simulated wholesale inside either counter factor; its responsibility has been distributed across the pair.

This gives a sharp contrast with the finite-state landscape. Bounded-factor decomposition remains within classical complexity classes for DFAs, including NP-completeness for commutative permutation DFAs [1]. For deterministic one-counter automata, by contrast, the same bounded formulation is already undecidable at $k = 2$.

A Transition tables

The tables below list the non-poisoning phase transitions. Every omitted non-core transition from a special zero state enters permanent-positive poison with update +1; every omitted transition at positive height enters permanent-positive poison with update 0. Core letters always translate at height zero and poison at positive height. At non-special factor cosets, query symbols overaccept at height zero and \$ resets to the special coset.

A.1 Checker 1

Mode	Input	Action
S_1 , zero	@	$(P_{p_0}, +1)$
P_p , positive	O_τ^1 , $\text{src}(\tau) = p$	$(S_1, -1)$
S_1 , raw	A_τ^1 , $\text{op}_1(\tau) = \text{inc}$	raw, +1
S_1 , raw	A_τ^1 , $\text{op}_1(\tau) = \text{skip}$	raw, 0
S_1 , zero raw	A_τ^1 , $\text{op}_1(\tau) = \text{zero}$	raw, 0
S_1 , positive raw	A_τ^1 , $\text{op}_1(\tau) = \text{dec}$	raw, -1
S_1 , raw	C_τ^1	$(P_{\text{tgt}(\tau)}, +1)$
P_p , positive	O_τ^2	$(K_\tau^A, 0)$
K_τ^A , positive	A_τ^2	$(K_\tau^C, 0)$
K_τ^C , positive	C_τ^2	$(P_{\text{tgt}(\tau)}, 0)$
P_{p_f} , positive	T_1	$(S_1, -1)$
S_1 , zero raw	Z_1	$(U_1, +1)$
U_1 , positive	T_2	$(U_2, 0)$
U_2 , positive	Z_2	$(U_3, 0)$
U_3 , height 1	\$	$(S_1, -1)$

A.2 Checker 2

Mode	Input	Action
S_2 , zero	@	$(B, +1)$
B , positive	O_τ^1	$(A_\tau, 0)$
A_τ , positive	A_τ^1	$(C_\tau, 0)$
C_τ , positive	C_τ^1	$(C_\tau, 0)$
C_τ , positive	O_τ^2	$(S_2, -1)$
S_2 , raw	A_τ^2 , $\text{op}_2(\tau) = \text{inc}$	raw, +1
S_2 , raw	A_τ^2 , $\text{op}_2(\tau) = \text{skip}$	raw, 0
S_2 , zero raw	A_τ^2 , $\text{op}_2(\tau) = \text{zero}$	raw, 0
S_2 , positive raw	A_τ^2 , $\text{op}_2(\tau) = \text{dec}$	raw, -1
S_2 , raw	C_τ^2	$(B, +1)$
B , positive	T_1	$(W_1, 0)$
W_1 , positive	Z_1	$(B, 0)$
B , positive	T_2	$(S_2, -1)$
S_2 , zero raw	Z_2	$(V_2, +1)$
V_2 , height 1	\$	$(S_2, -1)$

B Proof-audit checklist

The proof depends on several edge cases that are easy to conflate. The following points are all explicit in the construction above.

1. The target never stores both source counters; conjunction is enforced by the two factors at the zero anchor.
2. The factor budget is exactly two, and both factors have exactly $N/2$ states.
3. No hidden query-mode bit is assumed at zero height: raw zero and ordinary owner zero are the same physical configuration.
4. During every raw-zero interval, the complementary factor is positive and carries the lexical/sentinel witness.
5. Core interruption at raw zero cannot create a false common success: the zero factor translates, while the positive factor poisons.
6. A premature O^2 before C^1 is rejected by raw checker 1; hence checker 2 may safely reuse C_τ before and after C_τ^1 .
7. The terminal transition $B \xrightarrow{T_2, -1} S_2$ is explicit.
8. Immediate $\$$ is not a legal reset from a special zero state.
9. Successful reset renews the quotient invariant for repeated query episodes.
10. The halting/primalty argument is self-contained for arbitrary smaller deterministic one-counter factors, including ε -enabled factors, via the physical zero-cut lemma proved in Theorem 3.3.

References

- [1] I. Jecker, N. Mazzocchi, and P. Wolf, Decomposing permutation automata, *Journal of Computer and System Sciences* 156 (2026), Article 103721. doi:10.1016/j.jcss.2025.103721.
- [2] O. Kupferman and J. Mosheiff, Prime languages, in *Mathematical Foundations of Computer Science 2013*, LNCS 8087, Springer, 2013, pp. 607–618. doi:10.1007/978-3-642-40313-2_54.
- [3] M. L. Minsky, *Computation: Finite and Infinite Machines*, Prentice-Hall, 1967.
- [4] D. A. Spenner, Deciding DFA-primality is NP-hard, in *53rd International Colloquium on Automata, Languages, and Programming (ICALP 2026)*, LIPIcs 374, Article 192, 2026. doi:10.4230/LIPIcs.ICALP.2026.192.
- [5] P. Totzke, Prime DOCAs, Automata Exchange, Problem 20.03, 1 July 2020.
- [6] L. G. Valiant and M. S. Paterson, Deterministic one-counter automata, *Journal of Computer and System Sciences* 10(3) (1975), 340–350. doi:10.1016/S0022-0000(75)80005-5.